

Discrete Event Simulation

2026 Summer

kosuke imamura

What you learn

- How to simulate (what is needed)
- Tools to implement simulation software
 - Inter process communication (IPC)
(multi process programming, software hot swap)

How do we learn

- Lean by doing:
programming simple but applicable problems

Expected of you

- Complete assignments:
 1. Find a lost child
 2. Wait time
 3. Microwave oven
 4. Queuing system

Let's get started

- Before we get started, what is the advantages of simulation? That is, why do we want to simulate?

This N700 was
designed by ...



What have I simulated ?

- Commercial:
radar view, flight simulator, graphics co-processor (emulation), industrial computer printer, aluminum production optimization
- Academic:
performance analysis of a hardware unit
- Educational:
packet routing, correctness of concurrency, LED_display_panel

So what are the advantages of the above simulation?

What is DES?

- STATE

state/situation/condition

- EVENT

event at a point in time

- In other words,

event makes a state transit to another state

two kinds of event simulations

- next-event time progression
 - advance the clock to the next event
- incremental time progression
 - track small time slices for events to occur

Fundamentals (kind of ...)

- 0. Why simulation?
- 1. Define what you are trying to solve
- 2. Define state variables and events
- 3. Collect data
- 4. Of course, build the simulation system
- 5. Verify and validate
- 6. Analysis on the result

Assignment 1

- If a mother and child become separated and lost, both should stay put rather than move around. Moving simultaneously creates dynamic, unpredictable paths that drastically lower the mathematical probability of intersection and make tracking by rescuers or each other nearly impossible. (AI, [google.com/search](https://www.google.com/search))

Define

- What is the reason to simulate
-
- What you are solving
-
- Data to be collected

Define

- state variables
-
- event list
-
- **write a program!!**

Define

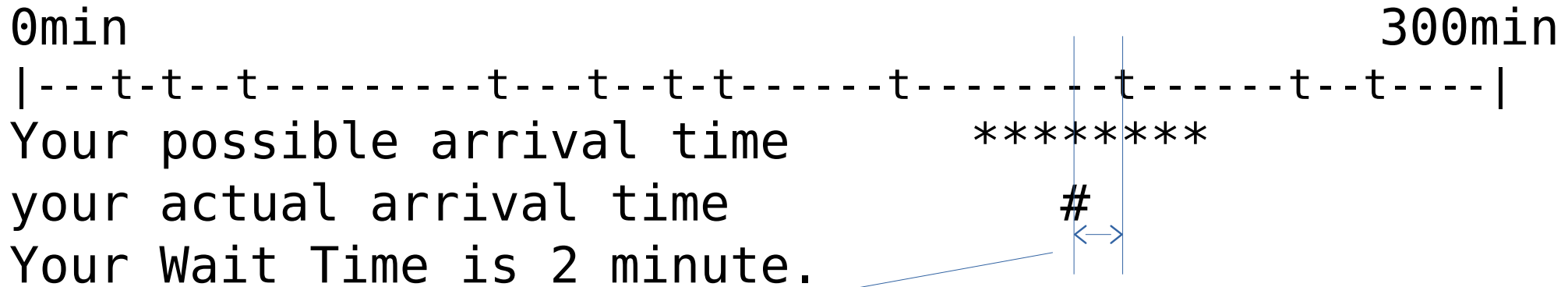
- purpose of this simulation
-
- event list
-
- write a program

Assignment 2

- Big snow fall.(brrr) IGR Iwate Ginga Railway's trains are off schedule because of heavy snow. Three trains leaves Iwate-Numakunai for Morioka, but the arrival times Numakunai for Morioka, but the arrival times are unpredictable. Now, you are taking the train to Morioka. Simulate the train wait time at Takizawa Station.

Big Picture (quality of random number)

Takizawa arrivals



DO NOT generate the train arrival times
using one hour duration, though random!!!!

Define

- What is the reason to simulate
-
- What you are solving
-
- Data to be collected

Define

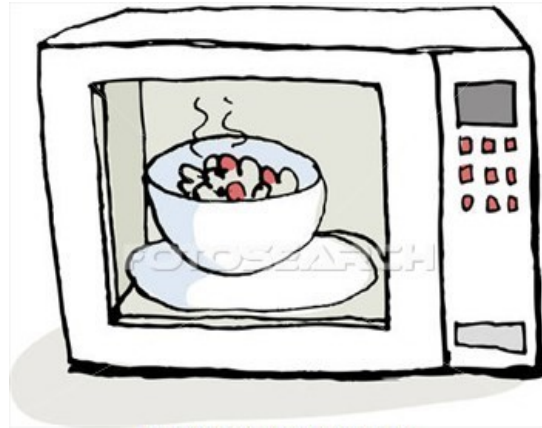
- state variables
-
- event list
-
- **write a program!!**

Define

- purpose of this simulation
-
- event list
-
- write a program

So, let's simulate a microwave oven.

- Purpose?
- In what occasion do we want to simulate a microwave oven?



Assignment 3

- microwave oven

Fundamentals (microwave oven)

- 0. Why simulation?

Fundamentals (microwave oven)

- 1. Define what you are trying to solve

Fundamentals (microwave oven)

- 2. Define state variables and events

Fundamentals (microwave oven)

- 3. Collect data

Fundamentals (microwave oven)

- 4. Of course, build the simulation system

Fundamentals (microwave oven)

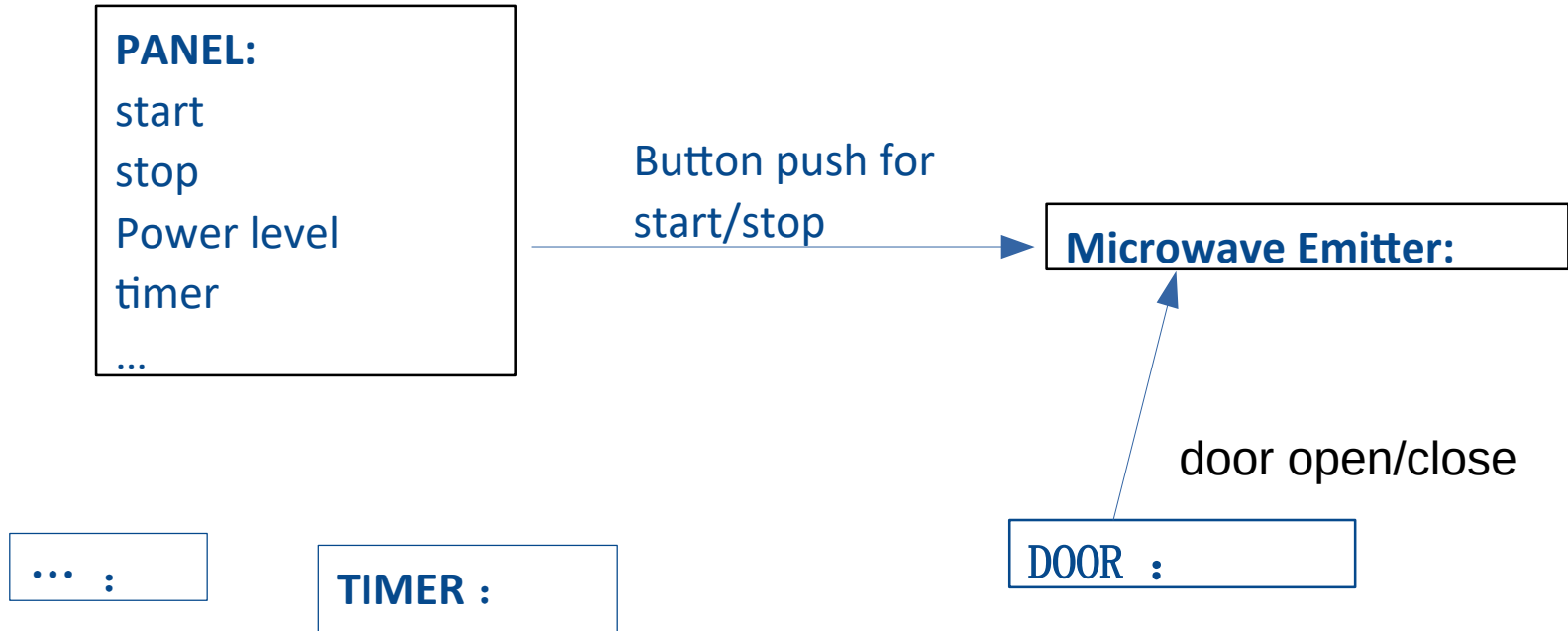
- 5. Verify and validate

Fundamentals (microwave oven)

- 6. Analysis on the result

IPC

Complete the control diagram.



Well, how about simulation methods

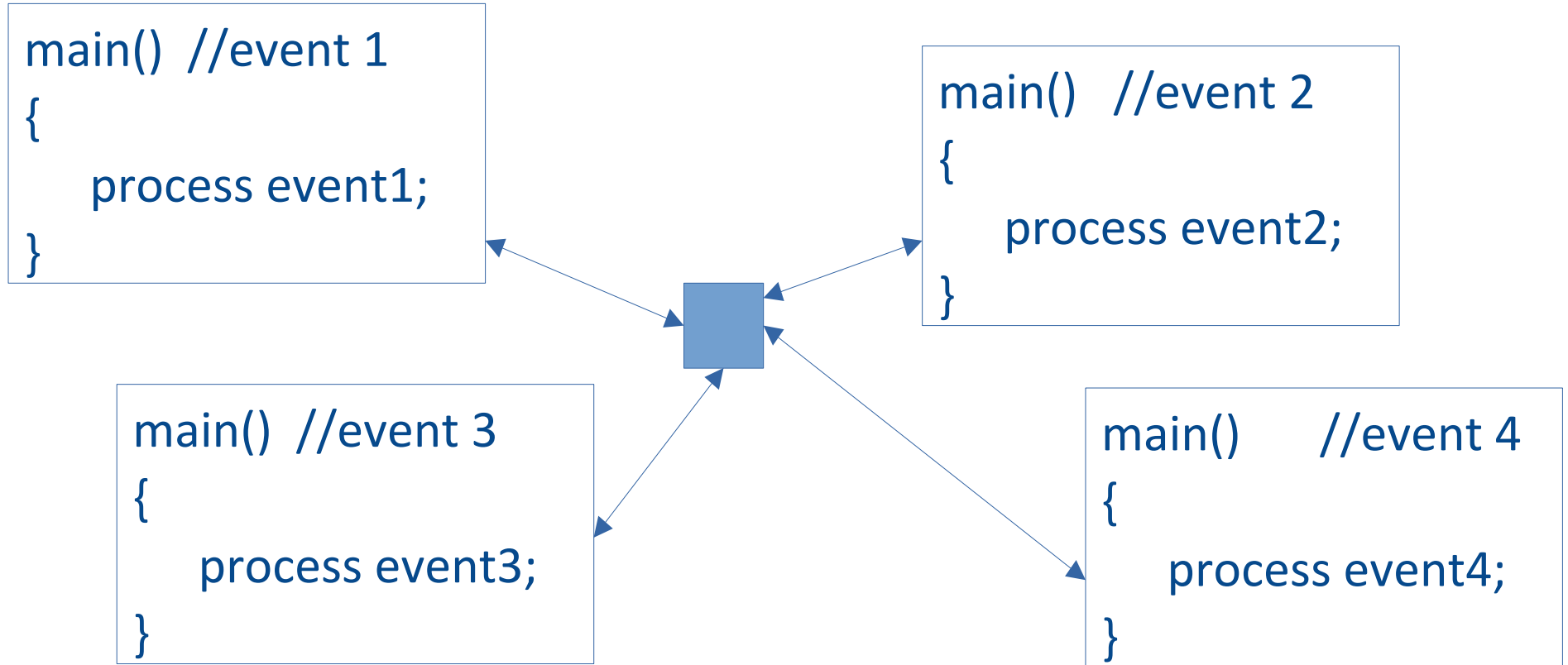
- 1 . Loop construct to process concurrent events
- 2 . Independent programs running separately and asynchronously.

Method 1

```
main()
{
    while(1)
    { Choose event i;
      process event i;
    }
}
```

State your opinion on this.

Method 2



State your opinion on this.

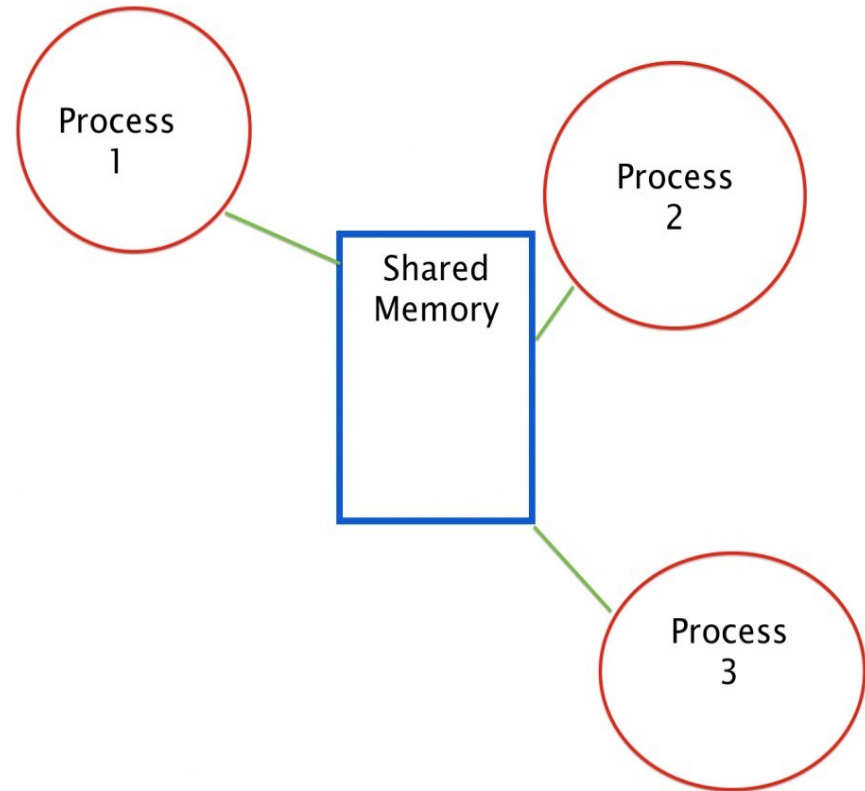
What is the shared memory

Well, well.....

Memory controlled by kernel

Multiple programs access the same memory

Exclusive access may needed if simultaneous write accesses



Microwave Devices

File:door.c Process-1

```
main()
{
//write to shared memory
DOOR
//DOOR = open;
}
```

File:emitter.c Process-2

```
main()
{
//read from shared memory DOOR
//if DOOR == open, stop emitting
}
```

File:timer.c Process-3

```
main()
{
//read from shared memory DOOR
//if DOOR == open, stop cooking timer
}
```

First, Header file `header.h`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/ipc.h>           //Inter Process Communication header
#include <sys/shm.h>           //Shared Memory header
enum op {SYNCH, DOOR, TIME, LIGHT, TURN_TBL, EMITTER,
KEYPAD };

// #define SYNCH 1
// #define DOOR 2    define is fine but    enum id {...} is convenient
```

Shared memory programming involves:

- `//making shared memory`
- `//attaching to shared memory`
- `//shared memory reading/writing`
- `//shared memory detach`
- `//deleting shared memory`

- **//Making shared memory**

shmget()

```
#include "header.h"
```

```
main() {
```

```
key_t key;
```

```
int shm_id;
```

```
int *shm;
```

```
key=0x520260A; //key= ftok(".", 'a' );
```

```
shm_id=
```

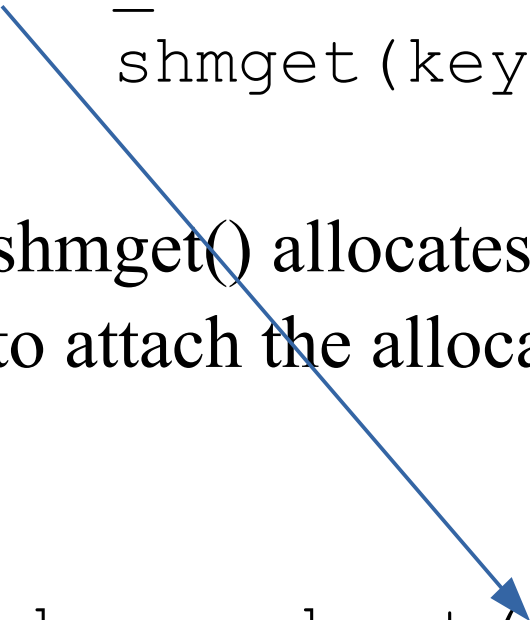
```
shmget(key, sizeof(int)*10, IPC_CREAT|0666);
```


//Shared memory attach shmat()

```
shm_id=  
    shmget(key, sizeof(int)*10, IPC_CREAT|0666);
```

shmget() allocates shared and returns the memory ID. Use the ID to attach the allocated memory:

```
shm = shmat(shm_id, 0, 0);    // char *shm;
```



- `//Shared memory reading/writing` `*shm`

After attaching, you can use it as if it were regular memories. Write may need exclusive access (semaphore)

```
* (shm+SYNCH) =0;
```

```
// enum op {SYNCH, DOOR, TIME, LIGHT,  
//          TURNTBL, EMITTER, KEYPAD };
```

//Shared memory detach shmdt()

- // 共有メモリデタッチ

```
shmdt(shm);
```

Shared Memory detach

Release memory pointed by shm.

//Deleting shared memory

- `shmctl(shmid,IPC_RMID,0)` can do it.
 - But, if process dies, then it can not do it.
 - Use IPC command to delete:
-
- `ipcs` ipc status コマンド
 - `ipcrm` ipc remove コマンド

```

#include "header.h"
main() {
key_t key; int shm_id; int *shm;

key=0x520260A;          //key = ftok(".", 'a' );
shm_id =
shmget(key, sizeof(int)*10, IPC_CREAT| 0666);

shm = shmat(shm_id,0,0);
*(shm+SYNCH) =0;

printf("all programs ready?\n"); getchar();
*(shm+SYNCH) =1;  // barrier synchronization

while(1)
{ if( *(shm+DOOR) == 0)
    printf("Door closed\n");
  else printf("Door open\n");
}
}

```

```

#include "header.h"
main() {
key_t key; int shm_id; int *shm;
int doorstat;

key=0x520260A;          //key = ftok(".", 'a' );
shm_id = shmget(key, sizeof(int)*10,
0666);

shm = shmat(shm_id,0,0);
while ( *(shm+SYNCH) == 0 );      //
barrier synch

while(1)
{
  if( doorstat == 0 )
    *(shm+DOOR) = 0;
  else *(shm+DOOR) = 1;
}
}

```

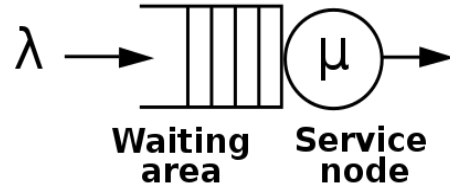
```
$ ipcs -m -m : memory -q : message Q
```

```
----- Shared Memory Segments -----
```

key	shmid	owner	perms	bytes	nattch
0x8f6e2129	98305	oracle	600	77694523	0
0x2f629238	65536	oracle	640	2736783360	35
0x00000000	32768	oracle	640	2736783360	0

dest

```
$ ipcrm shm 32768
```

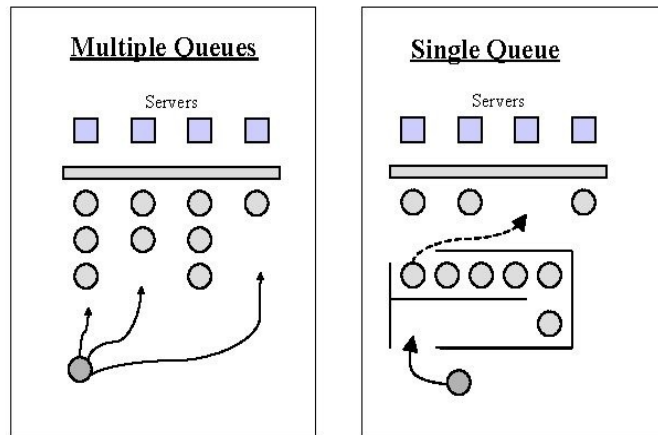


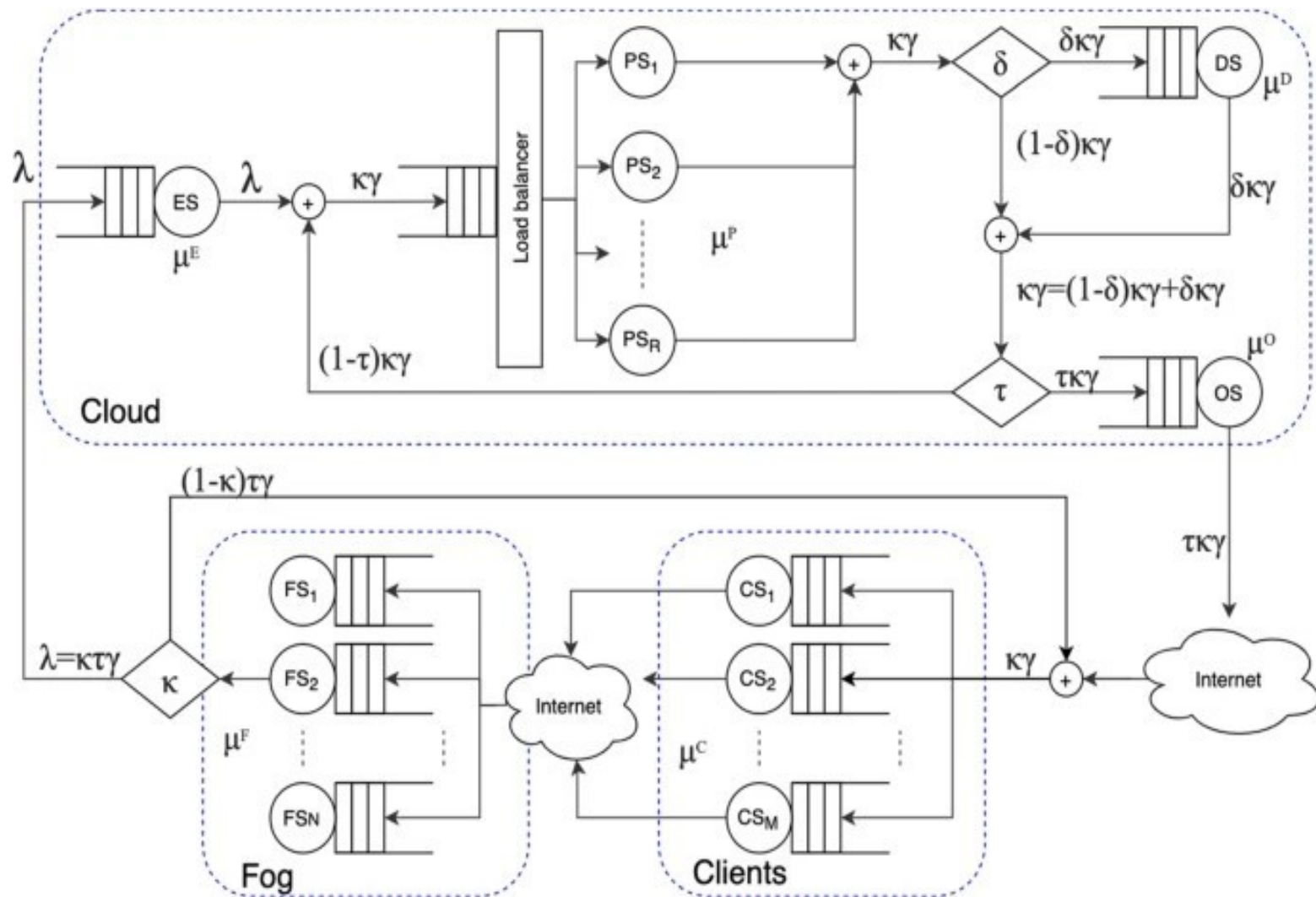
LITTLE'S LAW – IN STARBUCKS



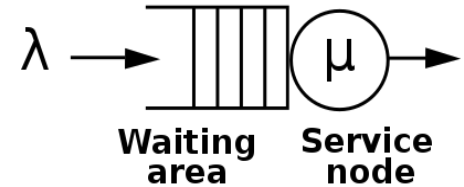
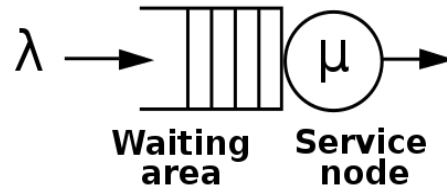
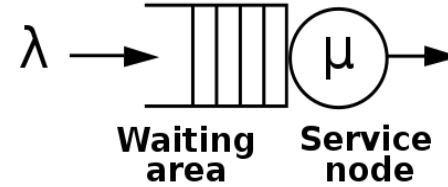
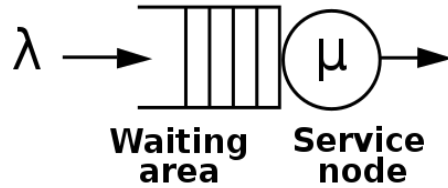
$$\text{Minutes until I get coffee} = \frac{\text{Number of people in line}}{\text{people served per minute}}$$

Example – Two Queue Configurations

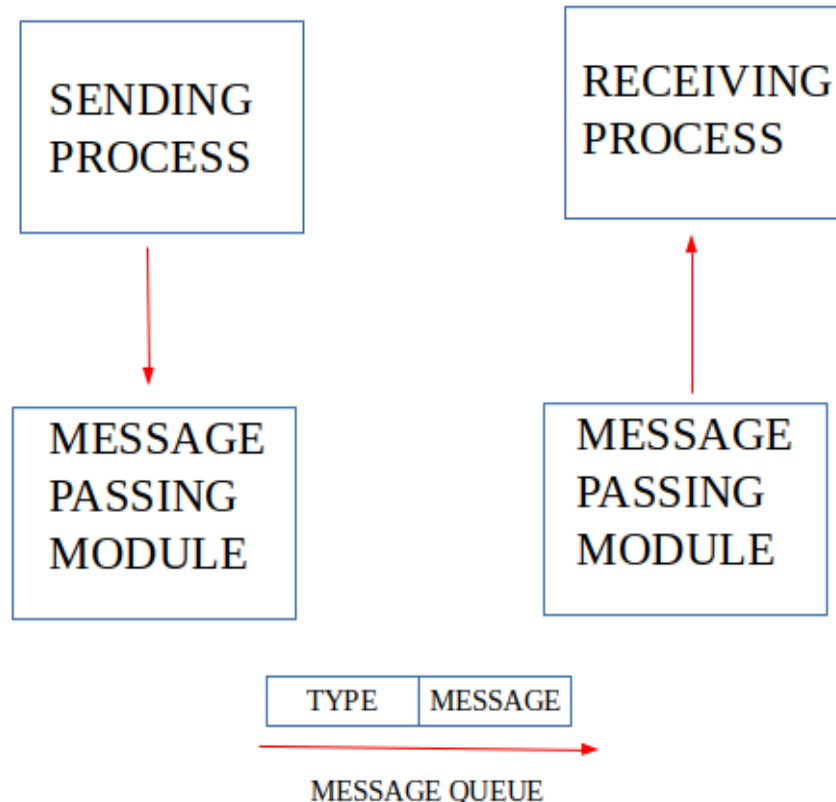




Let's first model the cafeteria.



We will use message queue for HW4.



Message Queues are very similar to Shared Memory.

- `ftok()`: is use to generate a unique key. または、直接。。。
- `msgget()`: either returns the message queue identifier for a newly created message queue or returns the identifiers for a queue which exists with the same key value.
- `msgsnd()`: Data is placed on to a message queue by calling `msgsnd()`.
- `msgrcv()`: messages are retrieved from a queue.
- `msgctl()`: It performs various operations on a queue. Generally it is use to destroy message queue.

Message Send, and Receive

```
struct msgbuf {  
    long mtype;           // Dequeued by the same type number msgrcv  
    char mtext[1];        // message body  
};
```

```
int msgsnd(int msgid, const void *msgp, size_t msgsz, int msgflg)
```

```
int msgrcv(int msgid, const void *msgp, size_t msgsz, long  
msgtype, int msgflg)
```

```

#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#define MAX 10

struct mesg_buffer {
    long mesg_type;
    char mesg_text[100];
} message;

int main()
{key_t key;
int msgid;
key = ftok(".", 65);

msgid = msgget(key, 0666 | IPC_CREAT);
message.mesg_type = 1;
fgets(message.mesg_text,MAX,stdin);

msgsnd(msgid, &message, sizeof(message), 0);
printf("Data send is : %s \n", message.mesg_text);
return 0;
}

```

```

#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>

struct mesg_buffer {
    long mesg_type;
    char mesg_text[100];
} message;

int main()
{key_t key;
int msgid;
key = ftok(".", 65);

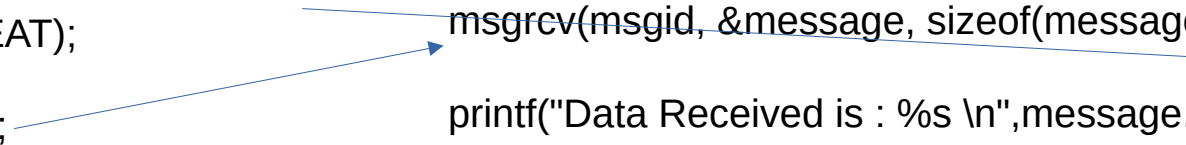
msgid = msgget(key, 0666 | IPC_CREAT);

msgrcv(msgid, &message, sizeof(message), 1, 0);

printf("Data Received is : %s \n",message.mesg_text);

    msgctl(msgid, IPC_RMID, NULL);
    return 0;
}

```



Checking the number of messages in a queue (simulating how many are in a waiting line)

```
int msgctl(int msqid, int cmd, struct msqid_ds *buf);  
msgctl(msqid, IPC_STAT, &buf) == -1) {  
    struct msqid_ds {  
        ...  
        msgqnum_t    msg_qnum; // # of msg in queue  
    };
```

Fundamentals (cafeteria)

- 0. Why simulation?
- 1. Define what you are trying to solve
- 2. Define state variables and events
- 3. Collect data
- 4. Of course, build the simulation system
- 5. Verify and validate
- 6. Analysis on the result

Fundamentals (cafeteria)

- 0. Why simulation?

Fundamentals (cafeteria)

- 1. Define what you are trying to solve

Fundamentals (cafeteria)

- 2. Define state variables and events

Fundamentals (cafeteria)

- 3. Collect data

Fundamentals (cafeteria)

- 4. Of course, build the simulation system

Fundamentals (cafeteria)

- 5. Verify and validate

Fundamentals (cafeteria)

- 6. Analysis on the result

